Handoff 06: Rate limits and security headers (stage 6: SEC-01, SEC-04)

Date: 2026-10-25 (America/Toronto)
Written by: Claude Code (implementer)
For: the human maintainer, and ChatGPT or Claude chat (reviewer) without repository access
Status: **in_review**: one pull request, not merged, not deployed. No content, XP or account-policy changes.

> **How to use this file:** upload it with `build.md`, `docs/plan/backlog.md` and `docs/plan/feedback-security.md`. It summarizes evidence; the pull request diff and CI results are the source of truth (`build.md` section 14).

Starter prompt for the chat assistant

```
You are reviewing Python Academy stage 6: rate limits on the API's write
endpoints and sign-in (SEC-01), and security headers with a
Content-Security-Policy in Report-Only mode (SEC-04). I'm attaching
build.md, the backlog, the feedback security plan and this handoff.
Check that a made-up X-Forwarded-For header cannot dodge or abuse the
limits, that the counters and logs hold no learner code or message text,
that the limits are loose enough for a classroom sharing one address,
and that the proposed enforce-mode policy is as strict as it can be
without breaking Pyodide. Explain at a first-year computer engineering
level.
```

1. Where we are

Stage	What	Status
1–3	Content format, migrations and ledgers, catalog UI	Merged and verified live (PRs #5, #7, #8)
4	Goals and recommendations	Merged (PR #9); owner's live check not yet recorded
5	Backlog, feature plans, feedback security	Merged (PR #10, documentation)
6	SEC-01 rate limits, SEC-04 security headers	**in_review**: this PR

2. Handoff record (build.md template)

Field	Value
Task	SEC-01: limit write endpoints and sign-in, return 429 with `Retry-After`. SEC-04: HSTS, `nosniff`, `Referrer-Policy`, framing blocked, page CSP in Report-Only mode with a tested enforce-mode policy. Close decision 19
Human goal	Stop scripted abuse (spam checks, invitation floods, sign-in hammering) before the feedback feature adds a new place to send text, and add browser-side protections without breaking Python in the browser
Repository and branch	`torontojohn/python-academy`, branch `sec/rate-limits-headers`
Base commit	`6d04ff0` (`main` after PR #10)
Deployment status	Not deployed; not merged
New dependencies	**None.** The limiter uses only Python's standard library; the tests use the existing pytest and Playwright

Files changed

File	Change
`backend/app/limits.py`	New. In-memory fixed-window limiter, the rule table, client-address logic, the 429 response
`backend/app/main.py`	Each limited route gets `dependencies=[Depends(limit('<rule>'))]`. API responses also get `Content-Security-Policy: default-src 'none'; frame-ancestors 'none'`, and HSTS in production
`backend/tests/test_limits.py`	New: 23 tests (exceeding each limit, normal use, counters and logs, client address)
`backend/tests/conftest.py`	New: empties the counters before and after every test so tests cannot push each other over a limit
`frontend/next.config.mjs`	HSTS; an enforced `frame-ancestors 'none'`; the page CSP in Report-Only mode. The worker's own enforced policy is unchanged and listed last so it still applies to the worker
`frontend/tests/academy.spec.ts`	Every test now fails if the page CSP reports a violation; new tests: Python runs with the headers on, the app refuses to be framed, a 429 shows its message and keeps the draft
`DEPLOYMENT.md`	Short section on limits and headers
`docs/plan/backlog.md`, `course-catalog.md`, `feedback-security.md`	Decision 19 closed; SEC-01 and SEC-04 in_review; new SEC-09 (CSP nonces); a correction to the feedback security plan (section 6 of this file)

3. SEC-01: rate limits

The limits

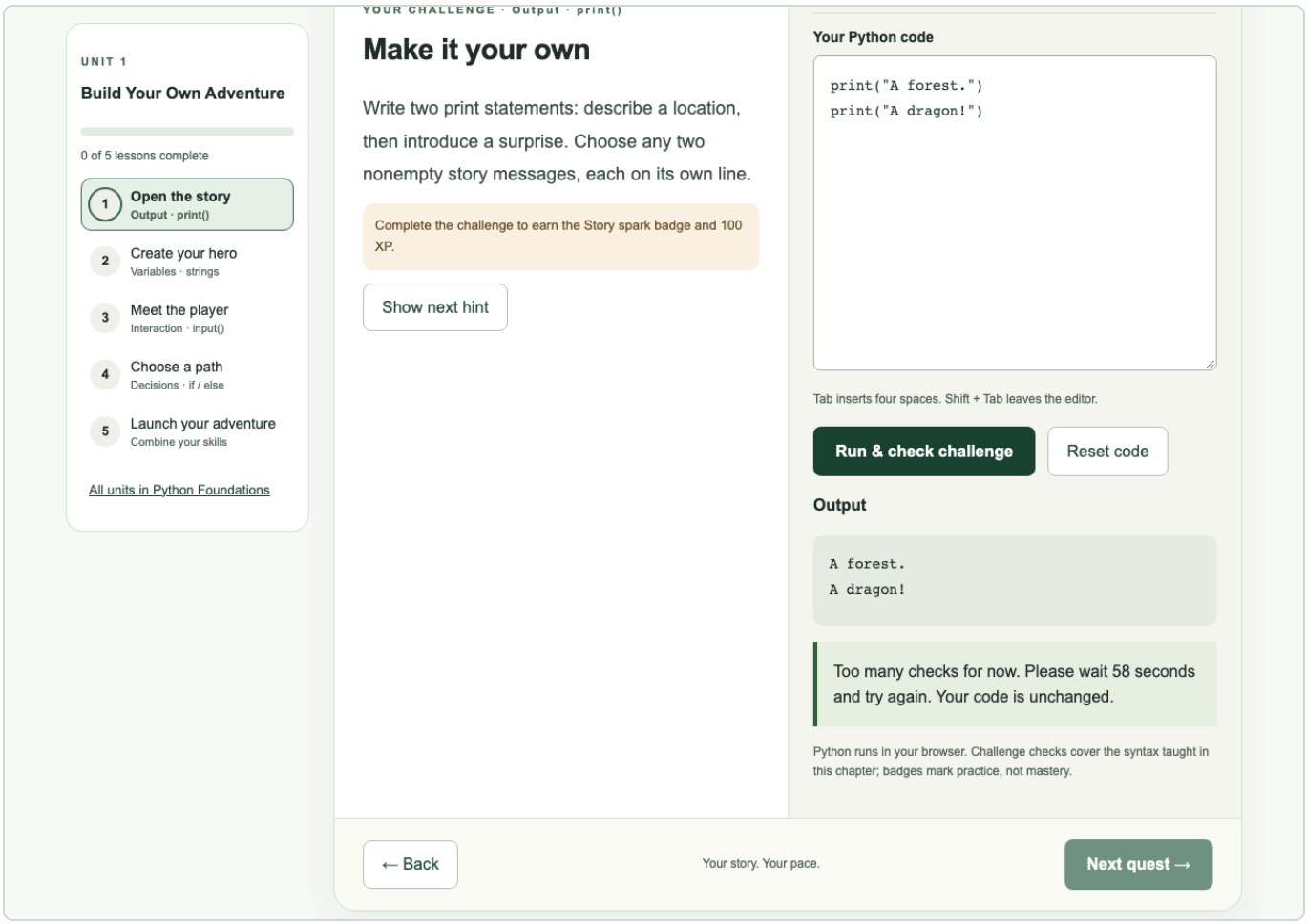
Rule	Endpoint	Signed in (per user)	Signed out (per address)
`check`	`POST /api/check` (preview checks)	20 per minute and 300 per hour	60 per minute and 600 per hour
`attempts`	`POST /api/attempts` (saved checks)	20 per minute and 300 per hour	60 per minute and 600 per hour (refused with 401 anyway)
`invitations`	`POST /api/invitations`	10 per hour and 30 per day	10 per hour
`goal`	`POST /api/goal`	10 per minute and 50 per day	30 per minute
`enroll`	`POST /api/enroll` (registration)	10 per 15 minutes	30 per 15 minutes
`signin`	`GET /auth/google`	always per address: 60 per 10 minutes	same
`signin_callback`	`GET /auth/google/callback`	always per address: 60 per 10 minutes	same

`enroll` was not on your list. I added it because it is a write endpoint where invitation codes are typed in. Logout is the only write endpoint without a limit, and a test fails if a new write endpoint is added without one.

Why these numbers: a busy learner who checks every 15 seconds for a whole hour stays under them (tested). Signed-out and sign-in limits are per address and looser, because a school or a family shares one address. Thirty learners on one address signing in and trying preview checks stay under the limits (tested).

What a refused request looks like

HTTP 429 with a `Retry-After` header (whole seconds), and a JSON message the app already shows in its notice area. For example: "Too many checks for now. Please wait 58 seconds and try again. Your code is unchanged." The draft stays in the editor and nothing is awarded.



The API log gets one line, `Rate limited: rule=check scope=ip`: no user ID, address, code or message text.

Where the counters live: memory, not the database

Choice: in the API process's memory. Each counter holds only a keyed hash of `u:<user id>` or `ip:<address>`, the start of its window, and a count. The hash key is random and new each time the process starts.

	Memory (chosen)	Database
Cost and complexity	No migration, no extra table, no extra database write on every check	A new table, a migration, a write per limited request on the Basic-256mb database, and a cleanup job
Privacy	Addresses and user IDs are never written to disk; everything is gone at restart	Hashed addresses stored on disk. The IPv4 range is small enough to reverse a plain hash, so it needs a keyed hash and deletion

	Memory (chosen)	Database
Restarts and deploys	**Counters reset**, so someone could get a fresh allowance after each deploy. Visitors cannot trigger deploys, so this costs at most one extra allowance per deploy	Survive restarts
More than one instance	**Breaks**: each instance would count separately. Today Render runs one API instance (Starter plan, no autoscaling). During a zero-downtime deploy, the old and new instances overlap for a moment	Correct with any number of instances
Memory use	Bounded: IPv6 addresses are grouped by /64, and above 50,000 counters the oldest are dropped (tested)	Bounded by cleanup

For a speed bump against scripts on one Render instance, memory is the simpler and more private choice. **If the API is ever scaled to two or more instances, move the counters to the database first** (noted in `DEPLOYMENT.md`).

The future feedback endpoint is different. Its daily and global limits must survive restarts, so it keeps the plan's own database counters (`feedback-security.md` section 7, keyed HMAC, no text).

Which address counts as the visitor

Browser → Render's proxy → Next.js → API. Render's proxy adds the visitor's address to `X-Forwarded-For`. I checked Next.js's proxy code: it passes the header through unchanged. A visitor can send their own `X-Forwarded-For`, but their made-up entries end up to the **left** of the entries the proxies add. The API therefore uses the **rightmost public address** in the header, falling back to the connection's address. A made-up entry can never choose someone else's counter or dodge your own (tested).

Not verified on Render. Render does not document its exact header format. Forum reports conflict, and the forum would not load for me to read the details. The live check below confirms it. If Render's proxy adds its own *public* address after the visitor's, every signed-out visitor would share one counter. That fails safe (too strict, never spoofable), and the live check would show it.

4. SEC-04: security headers

Headers now sent

Header	Pages (Next.js)	API (FastAPI)
`Strict-Transport-Security`	`max-age=31536000; includeSubDomains`	Same, in production only
`X-Content-Type-Options`	`nosniff` (already)	`nosniff` (already)

Header	Pages (Next.js)	API (FastAPI)
`Referrer-Policy`	`no-referrer` (already)	`no-referrer` (already)
Framing	`X-Frame-Options: DENY` (already) **and** enforced `Content-Security-Policy: frame-ancestors 'none'`	`Content-Security-Policy: default-src 'none'; frame-ancestors 'none'` (enforced; JSON needs nothing)
Page CSP	`Content-Security-Policy-Report-Only` (below)	—
Worker `/python-worker.js`	Its existing enforced policy, unchanged: Pyodide from `cdn.jsdelivr.net` only	—

Three things I found while building this:

- `frame-ancestors` **is ignored inside a Report-Only policy**, so the framing protection is sent as its own small enforced policy now. It cannot break Pyodide.
- **Next.js applies the last matching header rule** when two rules set the same header. The worker's rule is therefore listed last. Otherwise the page's `frame-ancestors` policy would have replaced the worker's policy.
- API responses pass through Next.js without Next.js's headers, so HSTS for `/api/…` comes from the API itself.

Violations found on the lesson page

Method: production build and the real local API. A strict baseline policy (`default-src 'self'; script-src 'self'; style-src 'self'; img-src 'self'; font-src 'self'; connect-src 'self'; worker-src 'self'; object-src 'none'; base-uri 'none'; form-action 'self'`) was set in Report-Only mode. A browser script then visited `/`, `/courses`, `/courses/py`, `/learn/hello`, `/preview?lesson=hello` and the challenge card, ran Python, got a check result, and ran the `input` lesson. It recorded every `securitypolicyviolation` event and console report. The 27 Playwright flows (signed-in student, parent, goal card, catalog, mobile) were then run with the same listener.

#	Directive	What was blocked	Where	Cause	Fix in the proposed policy
1	`script-src-elem`	Inline `<script>` elements, 2–4 per page (for example hashes `sha256-IeYfifqR…`, `sha256-OBTN3Riy…`, `sha256-Tt52LbKv…`)	Every page: `/`, `/courses`, `/courses/py`, `/learn/hello`, `/preview`	Next.js puts its page data and start-up code in inline scripts. Their content, and so their hashes, change with every page and every build	`script-src 'self' 'unsafe-inline'`. Hashes cannot list content that changes. Nonces need SEC-09

That was the only violation. No style, image, font, `connect-src` or `worker-src` violation occurred. Pyodide and its CDN load **inside the worker**, which has its own policy, so the page policy never sees them.

The page does not need `'unsafe-eval'` or the CDN.

Exact enforce-mode policy to switch to

```
Content-Security-Policy: default-src 'self'; script-src 'self' 'unsafe-inline'; style-src 'self'; img-src 'self'; font-src 'self'; connect-src 'self'; worker-src 'self'; object-src 'none'; base-uri 'none'; form-action 'self'; frame-ancestors 'none'
```

To switch, in `frontend/next.config.mjs` replace these two lines:

```
      {key:'Content-Security-Policy',value:"frame-ancestors 'none'"},
      {key:'Content-Security-Policy-Report-Only',value:PAGE_POLICY},
```

with this one:

```
      {key:'Content-Security-Policy',value:PAGE_POLICY+"; frame-ancestors 'none'"},
```

Already tested in enforce mode: I made exactly that change locally and rebuilt. All 27 Playwright tests passed, and Python ran and was checked against the real local API with no violations. Then I put the Report-Only version back.

What this policy does and does not do:

- It blocks scripts, styles, images and fonts from other sites. It blocks sending data anywhere except this app (`connect-src 'self'`), plugins, `<base>` tag tricks, forms posting elsewhere, and framing.
- Because of `'unsafe-inline'`, it does **not** block an injected inline `<script>`. React escapes all text, and the app never uses `dangerouslySetInnerHTML`, so there is no known way to inject one today. Closing this needs nonces: a Next.js proxy file adds a fresh random value to each page, and every page then renders per request. That is new backlog item **SEC-09**, which I propose as a dependency of the admin inbox.

5. Decision 19 closed

Parents never see the feedback button or any feedback; this follows from decision 23. It is recorded in `docs/plan/backlog.md` section 6, `docs/plan/course-catalog.md` section 9 (with the session record), and `docs/plan/feedback-security.md`. FB-01 no longer waits on it. Still open: 14, 15, 17, 20.

6. Correction to the feedback security plan

`feedback-security.md` said the SEC-04 policy "blocks inline script on admin pages". That would have been wrong, because Next.js needs `'unsafe-inline'`. The plan now says the policy blocks outside scripts and

outside data sending, and that blocking inline script needs SEC-09. It also says the feedback limits need database counters, not stage 6's in-memory ones.

7. Acceptance criteria

- Write endpoints `/api/check`, invitations, attempts and goal changes, plus sign-in, are limited: per user when signed in, per address when not. **Met** (plus enrolment).
- Refusals return 429, `Retry-After`, and a short plain message, and learner code and message text are never logged. **Met** (tested: counters and logs contain no code, address, user ID or email).
- Counter storage chosen, with the trade-off explained for Render, and no message text in counters. **Met** (section 3).
- Tests exceed each limit, and normal use stays under the limits. **Met** (23 backend tests).
- HSTS, `X-Content-Type-Options`, `Referrer-Policy` and `frame-ancestors` are on the main app. **Met.**
- The CSP is in Report-Only mode, every violation found is listed, and the exact enforce-mode policy is given. **Met** (section 4; enforce mode also tested).
- A Playwright test shows the lesson page still runs Python with the headers on. **Met.**
- Decision 19 closed in the plan and backlog. **Met.**
- No new dependencies; no content or XP changes. **Met.**

8. Tests run (exact commands and results)

From the repository root, on macOS with Python 3.12.14 (`backend/.venv`) and Node 24:

Command	Result
`cd backend && .venv/bin/python -m pytest -q`	`125 passed, 8 skipped` (the 8 are PostgreSQL tests; they need `TEST_POSTGRES_URL`)
Same, with `TEST_POSTGRES_URL` set to a local throwaway PostgreSQL 16 (pgserver)	`133 passed`
`cd backend && .venv/bin/python -m pytest -q tests/test_limits.py`	`23 passed`
`cd frontend && npx tsc --noEmit`	No errors
`cd frontend && npm run build`	Compiled successfully
`cd frontend && CI=1 npm run test:e2e`	`27 passed (38.9s)`. Every test also asserts no CSP violations
Same, built with the enforce-mode policy (section 4)	`27 passed (38.6s)`
Control: built with the stricter `script-src 'self'`	The header test **failed** as intended, with `script-src-elem blocked inline on /preview`. This proves the violation check works

Command	Result
Real local API: 61 `POST /api/check` from one address	60 × `200`, then `429`, `retry-after: 60`, and the message shown in the screenshot

CI runs the same backend tests, with its own PostgreSQL, and the Playwright suite on the pull request.

9. Mocks and unverified behaviour

- The Playwright tests mock the API, as before. The 429 test uses a mocked 429. The real 429 was checked against the local API (screenshot), and the backend tests use the real app.
- Render's `X-Forwarded-For` format is not verified (section 3). The live check covers it.
- The Google sign-in round trip was not run. The sign-in limits are tested with the routes called directly.

10. Questions for the owner

1. **SEC-09 before the admin inbox?** I propose making nonce-based CSP (SEC-09) a dependency of FB-06, because the admin inbox is where injected script would matter most. It is listed as proposed until you confirm.
2. **Limit numbers.** Are the values in section 3 right for how you expect testers and a classroom to use the app? They are one table in `backend/app/limits.py`.
3. **When to enforce.** I suggest one week of Report-Only with clean consoles during normal testing. Then switch with a one-line PR (section 4).

11. Live checks for you after merging

Merging redeploys both services. Use a terminal for the `curl` lines; nothing here needs a secret.

- [ ] **Deploy finished.** Both services show the new deploy as live in Render, and `https://python-academy-dji8.onrender.com/health` returns `{"status":"ok"}`.
- [ ] **Page headers.** `curl -sI https://python-academy-dji8.onrender.com/preview` shows `Strict-Transport-Security`, `Content-Security-Policy: frame-ancestors 'none'` and `Content-Security-Policy-Report-Only: default-src 'self'; …`.
- [ ] **Worker header unchanged.** `curl -sI https://python-academy-dji8.onrender.com/python-worker.js` shows `Content-Security-Policy: default-src 'none'; script-src 'unsafe-eval' https://cdn.jsdelivr.net; …`.
- [ ] **API headers.** `curl -s -D - -o /dev/null https://python-academy-dji8.onrender.com/api/lessons` shows `strict-transport-security` and `content-security-policy: default-src 'none'; frame-ancestors 'none'`.
- [ ] **Python still runs, with no CSP reports.** Open DevTools → Console, then go to a lesson challenge as a signed-in student. Run and check a correct answer, and confirm the badge and XP as before. The console should show **no** messages containing `Report Only` or `Content Security Policy`.

Repeat on the home page, `/courses`, a course page, the goal card ("Change goal") and the parent home. Send me any message you see, word for word.

- [ ] **Sign-in still works.** Sign out and sign back in with Google.
- [ ] **Normal use is not limited.** Do one whole lesson at a normal pace. You should never see "Too many …".
- [ ] **The limit works on the live site.** Run this, which sends 61 preview checks: `for i in $(seq 1 61); do curl -s -o /dev/null -w "%{http_code}\n" -H 'Origin: https://python-academy-dji8.onrender.com' -H 'Content-Type: application/json' -d '{"lesson_id":"hello","code":"print(1)","output":"1\n"}' https://python-academy-dji8.onrender.com/api/check; done | sort | uniq -c` Expect `60 200` and `1 429`. Your own address's signed-out checks are then paused for up to a minute.
- [ ] **A made-up address does not help.** Wait one minute, then run the same loop with `-H "X-Forwarded-For: 198.51.100.$i"` added. Expect `1 429` again: changing the header did not reset the counter.
- [ ] **Visitors are counted separately.** Straight after the previous check, on a phone using **mobile data (Wi-Fi off)**, open `https://python-academy-dji8.onrender.com/preview?lesson=hello&card=5` and run and check the challenge. It should work. If the phone also says "Too many checks", Render is sending one shared address. Tell me, and I will change how the address is chosen.
- [ ] **Logs hold no code.** In Render → API service → Logs, search for `Rate limited`. You should see `Rate limited: rule=check scope=ip` lines and nothing else about those requests: no code, email or address in that line. Render's own request log lines still show addresses, as before.
- [ ] **After about a week** of clean consoles, ask me for the one-line enforce-mode PR (section 4).

12. Next concrete action

The owner reviews this PR, merges it, and runs section 11. Then, following the backlog: record the stage 4 and stage 6 live checks, then SEC-03 (server-side logout) and SEC-09 (CSP nonces) before the admin inbox, then FB-01.